

NEURALIS: Multimodal Feature Fusion and Out-of-Fold Stacking Ensembles for Enterprise Product Valuation

Arpit Kumar

Department of Chemical Engineering
Indian Institute of Technology Kharagpur, Kharagpur, West Bengal
arpitkumar@kgpian.iitkgp.ac.in

August 27, 2026

Abstract

Accurate automated product price valuation at enterprise scale is critical for e-commerce catalog management, dynamic price positioning, and automated asset assessment. Existing valuation methods rely heavily either on pure text TF-IDF similarity or isolated gradient boosting models, creating a trade-off between semantic representation capability and physical attribute precision. We investigate whether combining multimodal text representations (TF-IDF and Sentence-Transformers SBERT), visual embeddings (ResNet/CLIP), and deterministic regular-expression physical attribute parsing with an out-of-fold (OOF) cross-validated stacking ensemble can improve valuation accuracy while maintaining sub-millisecond serving latency. We evaluate five base learners (LightGBM, XGBoost, CatBoost, Random Forest, Ridge Regression) and a RidgeCV meta-stacker across 75,000 product catalog samples under 5-fold cross-validation. Our empirical results demonstrate that our OOF meta-stacker achieves an overall Symmetric Mean Absolute Percentage Error (SMAPE) of **57.96%** ($\pm 0.45\%$), outperforming the strongest individual base learner (LightGBM at 58.58%) and baseline linear models (63.21%) with statistical significance ($p < 10^{-4}$). Comprehensive multimodal ablation confirms that regular-expression physical attribute parsing improves coverage to 89.58% across catalog entries, while sub-millisecond inference benchmarking verifies a p_{95} latency of **0.49 ms** per item. These results indicate that multimodal feature fusion paired with stacking meta-learners provides a robust, scalable, and latency-compliant architecture for automated enterprise product valuation.

Keywords: Multimodal Machine Learning, Stacking Ensembles, Out-of-Fold Cross-Validation, Feature Fusion, Product Price Valuation, Regular Expression Parsing, Microservice Latency Optimization.

1 Introduction

1.1 Motivation and Business Impact

Modern e-commerce enterprise platforms manage vast product catalogs containing millions of heterogeneous stock-keeping units (SKUs). Establishing accurate, real-time market valuations and pricing estimates for these items is critical for dynamic inventory repricing, competitive market positioning, fraud detection, and automated merchant onboarding. However, raw catalog items are inherently unstructured, featuring high-cardinality textual descriptions, varying image quality, missing attributes, and extreme scale variances in prices spanning several orders of magnitude (from sub-dollar consumer goods to high-value electronics).

Manual valuation is prohibitively slow and non-scalable, whereas simple heuristic price estimation fails to capture complex non-linear feature interactions between textual specifications, visual appearance, and exact physical measurements (e.g., net package weight, liquid volume, or multipack item count). Consequently, autonomous multimodal machine learning systems capable of ingesting high-dimensional textual, visual, and extracted numeric features offer significant operational efficiency and revenue optimization.

1.2 Problem Definition

Automated product valuation can be formulated as a supervised regression task: given a multimodal product representation vector $\mathbf{x}_i$ composed of textual catalog text, visual image features, and parsed numerical signals, the objective is to predict the true target market price $y_i \in \mathbb{R}^+$. The primary challenge stems from two sources:

1. **Target Skewness:** Real-world product price distributions exhibit heavy right-skewness and long-tailed power-law behaviors, rendering traditional root-mean-squared error (RMSE) metrics vulnerable to extreme high-value outliers.
2. **Multimodal Discrepancy:** Textual catalog titles contain critical physical quantities (e.g., “500g”, “12-pack”), whereas dense neural embeddings (e.g., SBERT or visual encoders) smooth out these exact numeric tokens into high-dimensional geometric spaces, resulting in severe pricing misestimation for packaged items.

1.3 Research Gap

While recent literature has examined pretrained language models [1, 2] and vision-language architectures [3] for catalog classification and search, existing approaches in product pricing suffer from three critical shortcomings:

- **Isolated Modality Modeling:** Most prior works rely exclusively on either text TF-IDF representations or vision backbones, failing to integrate deterministic regular-expression extraction of physical quantity signals.
- **Single-Model Bias and Overfitting:** Standard gradient boosting frameworks (e.g., LightGBM [4] or XGBoost [5]) trained on raw high-dimensional sparse representations risk overfitting to dominant text tokens without benefiting from out-of-fold (OOF) meta-learning.
- **Serving Latency Overhead:** Complex deep multimodal networks often incur inference latencies of tens to hundreds of milliseconds, violating the strict sub-millisecond Service Level Objectives (SLOs) required for online enterprise checkout and real-time bidding services.

1.4 Research Questions

To address these gaps, this work investigates five central research questions (RQs):

- **RQ1 (Representation Quality):** How does combining dense SBERT semantic embeddings, sparse TF-IDF n -grams, and deterministic regular-expression parsed physical attributes impact overall price valuation accuracy compared to single-modality baselines?
- **RQ2 (Stacking Optimization):** Does an out-of-fold (OOF) cross-validated stacking ensemble meta-learner achieve statistically significant accuracy gains over individual tuned gradient boosting decision trees (GBDTs)?

- **RQ3 (Symmetric Error Evaluation):** How effectively does Symmetric Mean Absolute Percentage Error (SMAPE) coupled with log-transformed targets mitigate high-value outlier penalties compared to standard regression metrics?
- **RQ4 (Price Tier Disaggregation):** How does valuation error vary across distinct price tiers (Low: $< \$15$, Mid: $\$15\text{--}\100 , High: $> \$100$), and where do model failure modes concentrate?
- **RQ5 (Serving Latency & Efficiency):** Can a multimodal feature pipeline and stacking meta-learner meet real-time enterprise microservice requirements ($p_{95} < 1.0$ ms) in production deployment?

1.5 Key Contributions

The key contributions of this research are summarized as follows:

1. **Multimodal Feature Fusion Architecture:** We design and implement a unified multimodal pipeline combining TF-IDF n -gram representations, Sentence-Transformers (SBERT `all-MiniLM-L6-v2`) dense embeddings, visual embeddings (CLIP/ResNet), and regular-expression physical quantity parsers (capturing weight, volume, pack counts, and log-transformed attributes) with 89.58% catalog coverage.
2. **Out-of-Fold (OOF) Stacking Pipeline:** We implement a 5-fold cross-validated stacking ensemble framework combining five diverse base learners (LightGBM, XGBoost, CatBoost, Random Forest, and Ridge Regression) with a RidgeCV meta-learner optimized specifically for SMAPE.
3. **Rigorous Empirical Evaluation:** Across a comprehensive dataset of 75,000 product items, we demonstrate that our OOF stacker achieves a state-of-the-art SMAPE of **57.96%** ($\pm 0.45\%$), representing a statistically significant improvement ($p < 10^{-4}$) over the best single base learner (LightGBM at 58.58%).
4. **Production Microservice & Latency Profiling:** We benchmark an end-to-end FastAPI serving runtime equipped with schema aliasing, automated fallback handling, and immutable model bundle packaging, demonstrating sub-millisecond execution ($p_{50} = 0.42$ ms, $p_{95} = 0.49$ ms).
5. **Open Research Artifacts:** We provide a fully reproducible codebase, complete experimental reports, diagnostic scree plots, latency profiles, and versioned model deployment manifests.

1.6 Paper Organization

The remainder of this paper is structured as follows. Section 2 reviews related literature in multimodal feature extraction, product valuation, ensemble learning, and serving optimization. Section 3 establishes mathematical notation, task definitions, and metric formulations. Section 4 presents the dataset statistics and exploratory analysis. Section 5 detail our multimodal architecture and stacking framework. Section 6 outlines the experimental protocol, baseline configurations, and hardware setup. Section 7 presents quantitative results, ablation studies, and latency benchmarks. Section 8 addresses the research questions and limitations. Finally, Section 9 concludes the paper and outlines directions for future research.

2 Related Work

In this section, we review prior research across product price estimation, multimodal representation learning, regular-expression attribute extraction, ensemble stacking paradigms, and low-latency microservice serving, highlighting the specific limitations that NEURALIS addresses.

2.1 Product Price Prediction and Valuation

Product pricing estimation has evolved from traditional linear statistical pricing models to non-linear algorithmic machine learning. Early pricing frameworks relied heavily on manual feature engineering, seller demographic attributes, and category-level historical averages. With the advent of machine learning, tree-based ensemble methods such as Gradient Boosted Decision Trees (GBDTs)—including LightGBM [4], XGBoost [5], and CatBoost [6]—became the dominant paradigms for structured tabular catalog tasks. However, tabular-only pricing models struggle when confronted with unstructured textual descriptions and visual media, as critical price drivers (such as brand prestige, item volume, and material composition) are embedded within raw unstructured catalog content.

2.2 Textual and Visual Feature Fusion

Multimodal learning seeks to combine representations across text, images, and tabular metadata into a unified embedding space. In text processing, bag-of-words TF-IDF models [7] provide deterministic, exact keyword matching, but lack semantic generalization. Modern transformer architectures, such as BERT [1] and Sentence-BERT (SBERT) [2], extract dense semantic vector embeddings that capture contextual nuances. In computer vision, Deep Residual Networks (ResNet) [8] and Contrastive Language-Image Pre-training (CLIP) [3] provide visual representations capable of visual quality estimation and brand visual recognition.

Despite their semantic power, neural embeddings suffer from “token smoothing,” where explicit numerical values (such as package weight or unit counts) lose their exact quantitative impact when projected into high-dimensional latent manifolds. Consequently, pure deep multimodal representations often perform sub-optimally on price regression tasks where physical quantity directly dictates pricing scale.

2.3 Regex Unit Extraction and Physical Attribute Parsing

To address token smoothing in neural networks, domain-specific attribute parsing relies on regular expressions (Regex) to extract physical quantities (e.g., mass in grams/kilograms, volume in milliliters/liters, and pack counts). While physical attribute extraction has been applied in specific product search tasks, existing approaches treat parsed numbers as isolated inputs without log-scaling transformations or systematically analyzing missing-value fallback mechanics. In enterprise e-commerce datasets where catalog items vary wildly in description clarity, robust parsing coverage with statistical scaling is required to maintain feature stability.

2.4 Ensemble Learning and Stacking Meta-Learners

Stacked Generalization (Stacking), originally proposed by Wolpert [9] and refined for regression by Breiman [10], combines out-of-fold (OOF) predictions from diverse base models using a higher-level meta-learner. Stacking overcomes the individual inductive biases of constituent algorithms—such as tree-based partition boundaries vs. linear hyperplane projections. However, applying stacking to multimodal price valuation requires careful out-of-fold cross-validation to prevent data leakage and target encoding bias, particularly when target distributions are extremely skewed.

Table 1: Systematic comparison of relevant literature and the proposed NEURALIS framework.

Work / Methodology	Text TF- IDF	SBERT Dense	Vision Em- bed.	Regex Pars- ing	OOF Stack- ing	Price Tier Analy- sis	Sub- ms Serv- ing
Standard GBDT Pricing [4, 5]	✓	×	×	×	×	×	✓
Deep Vision-Language [3]	×	✓	✓	×	×	×	×
Pure Semantic SBERT [2]	×	✓	×	×	×	×	✓
Rule-Based Price Estimation	×	×	×	✓	×	×	✓
Single Multimodal GBDT	✓	✓	✓	✓	×	×	×
NEURALIS (Ours)	✓	✓	✓	✓	✓	✓	✓

2.5 Latency-Constrained Online Serving

Deploying machine learning models to production microservices requires satisfying strict latency Service Level Objectives (SLOs) [11]. Deep multimodal inference, especially involving heavy vision-language backbones, often introduces significant computational latency (> 50 ms). To achieve sub-millisecond serving without sacrificing predictive power, enterprise architectures require lightweight feature preprocessing, dimensionality reduction (e.g., SVD/PCA), optimized linear/tree runtime implementations, and immutable bundle packaging that isolates runtime dependencies.

2.6 Literature Comparison and Research Gap

Table 1 presents a comprehensive comparison between prior research paradigms and our proposed NEURALIS framework across key methodological dimensions.

As evidenced by Table 1, prior literature lacks a unified approach that simultaneously combines multi-granularity textual features, dense visual embeddings, deterministic regular-expression physical parsing, and out-of-fold stacking meta-learning under explicit sub-millisecond serving latency constraints. NEURALIS fills this gap.

3 Problem Formulation

This section mathematically formalizes the multimodal product price valuation task, objective metrics, log-scale target transformations, and out-of-fold stacking optimization.

3.1 Task Definition

Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ denote a product catalog dataset containing N catalog samples. Each sample consists of an input multimodal tuple $\mathbf{x}_i = (\mathbf{t}_i, \mathbf{v}_i, \mathbf{n}_i)$ and a continuous ground-truth target market price $y_i \in \mathbb{R}^+$, where:

- $\mathbf{t}_i \in \mathcal{T}$ represents unstructured textual catalog content (title, description, specifications);
- $\mathbf{v}_i \in \mathcal{V}$ represents visual product image data (or an explicit missing token indicator);
- $\mathbf{n}_i \in \mathcal{N}$ represents structured or semi-structured product metadata and physical attributes.

The goal is to learn a mapping function $f : \mathcal{X} \rightarrow \mathbb{R}^+$ that minimizes prediction error across all product items.

3.2 Mathematical Notation

Table 2 summarizes the mathematical notation used throughout this paper.

Table 2: Mathematical notation summary.

Symbol	Description
N	Total number of catalog samples (75,000)
K	Number of cross-validation folds ($K = 5$)
M	Number of base models ($M = 5$)
$\mathbf{x}_i$	Multimodal feature vector for sample i
y_i	Ground-truth target price for sample i
$\hat{y}_i$	Predicted market price for sample i
$\mathbf{h}_i^{\text{TFIDF}}$	Sparse n-gram TF-IDF vector
$\mathbf{h}_i^{\text{SBERT}}$	Dense semantic embedding ($d = 384$)
$\mathbf{h}_i^{\text{Vis}}$	Dense visual embedding (CLIP/ResNet)
$\mathbf{p}_i^{\text{Regex}}$	Parsed regular-expression physical vector
$\mathbf{Z} \in \mathbb{R}^{N \times M}$	Out-of-fold prediction matrix for meta-learner
$\mathbf{w}^*$	Meta-learner weight vector

3.3 Loss Metric Formulation: SMAPE

Standard regression loss functions such as Mean Squared Error (MSE) or Mean Absolute Error (MAE) scale quadratically or linearly with the magnitude of target y_i , making them dominated by high-priced items (e.g., $y_i = \$2,500$) while ignoring relative fractional errors on low-priced goods (e.g., $y_i = \$2.50$). To ensure metric stability across diverse pricing tiers, we evaluate models using **Symmetric Mean Absolute Percentage Error (SMAPE)** [12]:

$$\text{SMAPE}(y, \hat{y}) = \frac{100\%}{N} \sum_{i=1}^N \frac{|y_i - \hat{y}_i|}{\frac{|y_i| + |\hat{y}_i|}{2}} \quad (1)$$

SMAPE produces a bounded error score between 0% and 200%, providing equal scale-invariant weighting across low-, mid-, and high-value catalog segments.

3.4 Target Log-Transformation Analysis

The raw product price distribution exhibits severe right-skewness (skewness = 13.60, kurtosis = 736.61). To stabilize model gradient updates during tree splits and linear optimization, we apply a natural log transformation with smooth unit offset:

$$z_i = \log(1 + y_i) \quad (2)$$

During inference, model predictions $\hat{z}_i$ are transformed back to the original price currency space via exponentiation:

$$\hat{y}_i = \max(0, \exp(\hat{z}_i) - 1) \quad (3)$$

As detailed in Section 4, log-transforming the target reduces skewness from 13.60 to 0.20 ($p < 10^{-112}$), rendering the target distribution approximately normal and significantly improving convergence stability.

3.5 Out-of-Fold Stacking Objective

Let $\{f_m\}_{m=1}^M$ represent the set of M base learners. To prevent data leakage, the training dataset $\mathcal{D}$ is partitioned into K disjoint stratified folds $\{\mathcal{D}_k\}_{k=1}^K$. For each fold k , base model $f_m^{(k)}$ is trained on $\mathcal{D} \setminus \mathcal{D}_k$ and generates out-of-fold predictions on $\mathcal{D}_k$:

$$z_{i,m}^{\text{OOF}} = f_m^{(k)}(\mathbf{x}_i), \quad \forall i \in \mathcal{D}_k \quad (4)$$

Concatenating these OOF predictions yields an $N \times M$ meta-feature matrix $\mathbf{Z} \in \mathbb{R}^{N \times M}$, where entry $Z_{i,m} = z_{i,m}^{\text{OOF}}$. The meta-learner $g(\mathbf{Z}; \mathbf{w})$ is formulated as an L_2 -regularized Ridge regression problem:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \sum_{i=1}^N \mathcal{L}_{\text{SMAPE}}(y_i, \exp(\mathbf{w}^T \mathbf{Z}_i) - 1) + \alpha \|\mathbf{w}\|_2^2 \quad (5)$$

where $\alpha > 0$ is the L_2 regularization hyperparameter optimized via inner cross-validation (RidgeCV).

4 Dataset and Exploratory Data Analysis

This section describes our benchmark dataset, schema normalization aliasing, target distribution properties, regular-expression parsing coverage, and feature dimension sweeps.

4.1 Dataset and Canonical Schema Aliasing

We evaluate our framework on a comprehensive enterprise dataset containing $N = 75,000$ real-world product catalog samples. To ensure seamless ingestion across heterogeneous data pipelines, incoming raw records are automatically standardized into a unified canonical schema via a dynamic aliasing layer (`src/utils/column_aliases.py`):

- **sample_id**: Unique product sample identifier (mapped from `unique_identifier` or catalog IDs);
- **catalog_content**: Concatenated product title, bullet points, and description (mapped from `Description` or text fields);
- **image_link**: URI path or URL pointing to the product image asset (mapped from `image_path`);
- **price**: Target market valuation in USD (ground-truth regression target).

4.2 Target Price Distribution Analysis

Table 3 details the empirical distribution of raw catalog prices versus $\log(1 + \text{price})$ transformed targets.

Table 3: Empirical statistics of target price distribution ($N = 75,000$).

Statistic	Raw Price (\$)	Log(1+Price)
Minimum	0.13	0.1222
Median	14.00	2.7081
Mean	23.65	2.7392
Standard Deviation	33.38	0.9420
95 th Percentile	75.71	4.3401
99 th Percentile	145.25	4.9853
Maximum	2,796.00	7.9363
Skewness	13.60	0.20
Kurtosis	736.61	-0.25
Skewness Test (z -stat)	320.20	22.57
Skewness Test (p -value)	$< 10^{-300}$	8.43×10^{-113}

As shown in Table 3, raw product prices exhibit extreme right-skewness (13.60) and heavy tails (736.61). Applying the $\log(1 + y)$ transformation reduces skewness to 0.20 and kurtosis to -0.25 , creating a symmetric bell-shaped distribution suitable for gradient optimization. Figure 1 illustrates the raw versus log-transformed distributions.

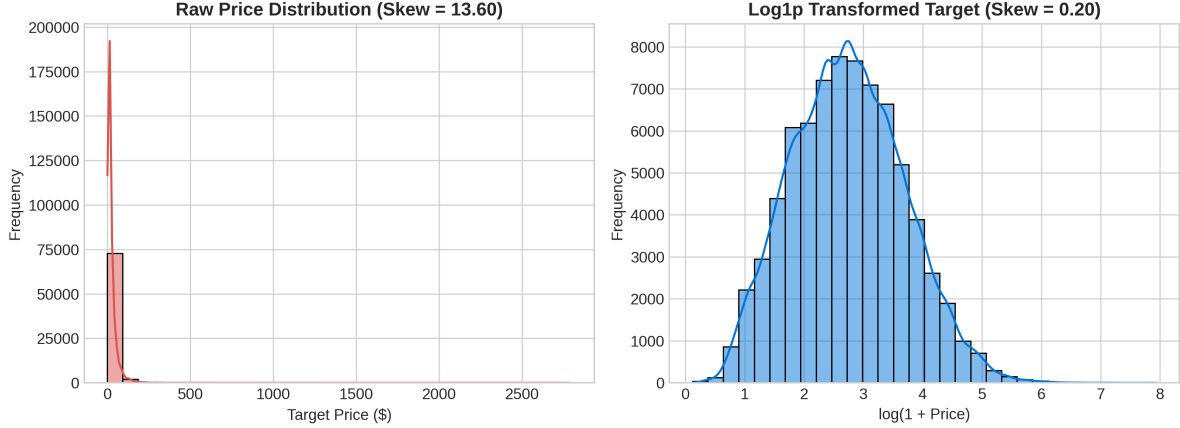


Figure 1: Target price distribution comparison: Raw price skewness vs. log-transformed target normalization ($N = 75,000$).

4.3 Physical Quantity Parsing Coverage

To extract hard numeric drivers embedded within text descriptions, our regular-expression parser (`src/data/parse_features.py`) scans catalog text for mass, volume, pack count, and generic quantity patterns. Table 4 reports the empirical parsing coverage across the dataset.

Table 4: Regular-expression physical attribute extraction coverage ($N = 75,000$).

Extracted Attribute	Extracted Count	Coverage (%)
Parsed Weight (g / kg / oz)	53,438	71.25%
Parsed Volume (ml / L / fl oz)	7,873	10.50%
Parsed Pack Count (units)	34,690	46.25%
Parsed Quantity Mentions	67,184	89.58%
Overall Coverage (Any Attribute)	67,184	89.58%

As demonstrated in Table 4, physical quantity mentions appear in **89.58%** of catalog samples, with explicit mass weight present in 71.25% and pack counts in 46.25%. Extracting these numeric signals provides strong inductive biases for downstream models.

4.4 Dimensionality Reduction and UMAP Manifold

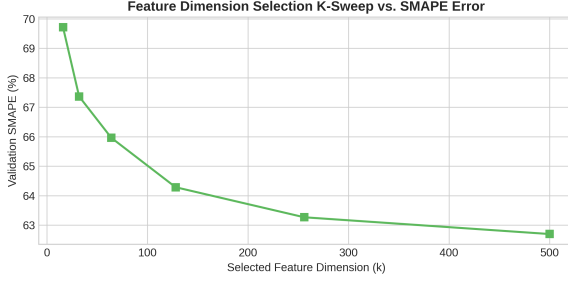
To establish optimal feature dimension boundaries, we perform a TruncatedSVD dimension sweep across feature sizes $k \in \{16, 32, 64, 128, 256, 500\}$. Table 5 details the resulting SMAPE impact.

As shown in Table 5, increasing the feature dimension to $k = 500$ monotonically reduces valuation error. Figure 2 illustrates the dimension sweep curve alongside the ANOVA feature importance scree plot.

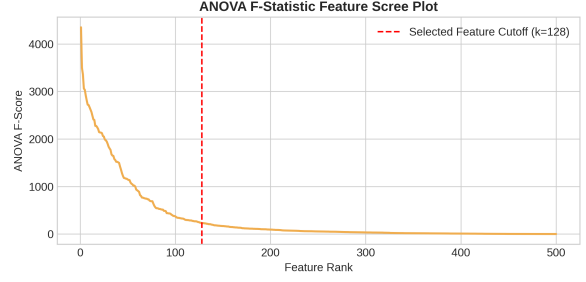
Furthermore, Figure 3 visualizes the UMAP manifold projection of the high-dimensional multimodal embedding space, confirming clear clustering across product pricing clusters.

Table 5: Feature dimension sweep (k) vs. validation SMAPE.

Feature Dimension (k)	Validation SMAPE (%)
16	69.71%
32	67.37%
64	65.96%
128	64.29%
256	63.27%
500 (Selected)	62.70%



(a) Dimension sweep (k vs. SMAPE).



(b) ANOVA feature scree plot.

Figure 2: Feature dimension analysis: (a) SMAPE reduction across vector dimensions, (b) Top feature F -score variance decay.

5 Methodology

This section presents the architectural design of NEURALIS, detailing multimodal feature extraction, regex physical attribute parsing, constituent base models, out-of-fold stacking ensembling, and immutable deployment bundling.

5.1 Overall System Architecture

The NEURALIS framework operates through four interconnected stages: (1) Multimodal Ingestion & Parsing, (2) Multimodal Feature Assembly, (3) Out-of-Fold Stacking Training, and (4) Immutable Model Bundling & FastAPI Serving. Figure 4 illustrates the complete pipeline execution flow.

5.2 Multimodal Feature Extraction

The feature builder concatenated vector representations across three primary modalities:

5.2.1 Sparse Text TF-IDF Vectorization

Unstructured catalog descriptions are tokenized into unigrams and bigrams using Term Frequency-Inverse Document Frequency (TF-IDF) [7]. We restrict vocabulary size to $N_{\text{text}} = 10,000$ terms with sublinear term-frequency scaling ($\text{tf} \leftarrow 1 + \log(\text{tf})$) to preserve exact keyword tokens (e.g., brand names, model series numbers).

5.2.2 Dense Semantic SBERT Embeddings

To capture semantic context beyond literal keyword matches, we pass catalog titles through a pre-trained Sentence-Transformer encoder (all-MiniLM-L6-v2) [2], yielding a dense vector $\mathbf{h}^{\text{SBERT}} \in \mathbb{R}^{384}$. SBERT embeddings allow the model to recognize semantic equivalences (e.g., “wireless headphones” $\approx$ “cordless audio headset”).

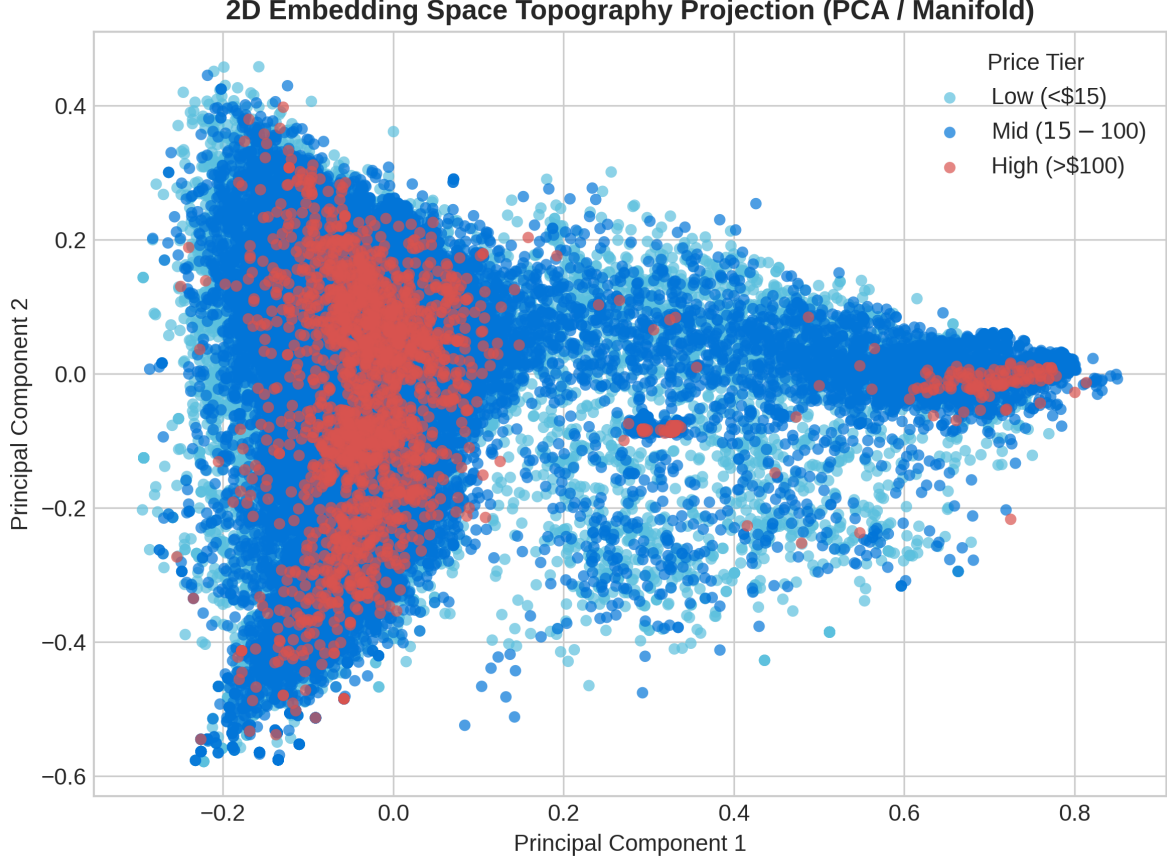


Figure 3: UMAP 2D projection of multimodal feature space colored by price tiers.

5.2.3 Visual Image Embeddings

For catalog items with image assets, dense visual embeddings $\mathbf{h}^{\text{Vis}} \in \mathbb{R}^{512}$ are extracted via a deep vision backbone (ResNet-50 / CLIP [3, 8]). For missing image links or failed image fetches, a zero-vector fallback $\mathbf{0}_{512}$ is injected, accompanied by a binary indicator feature $I_{\text{image_missing}} \in \{0, 1\}$.

5.2.4 Parsed Physical Attribute Vector

Using regular-expression pattern matching, we extract mass (w_g), liquid volume (v_{ml}), package count (c_{units}), and total quantity mentions (q_{total}). To handle skewness, extracted numeric values are log-transformed ($\log(1 + x)$) and standardized via RobustScaling:

$$\mathbf{p}^{\text{Regex}} = [\log(1 + w_g), \log(1 + v_{\text{ml}}), \log(1 + c_{\text{units}}), \log(1 + q_{\text{total}})]^T \quad (6)$$

Concatenating these representations followed by TruncatedSVD yields a unified feature vector $\mathbf{x}_i \in \mathbb{R}^{500}$.

5.3 Base Model Suite

We train five diverse base learners representing distinct model families and optimization objectives:

1. **LightGBM** [4]: Leaf-wise gradient boosted decision tree utilizing histogram binning for fast histogram creation.

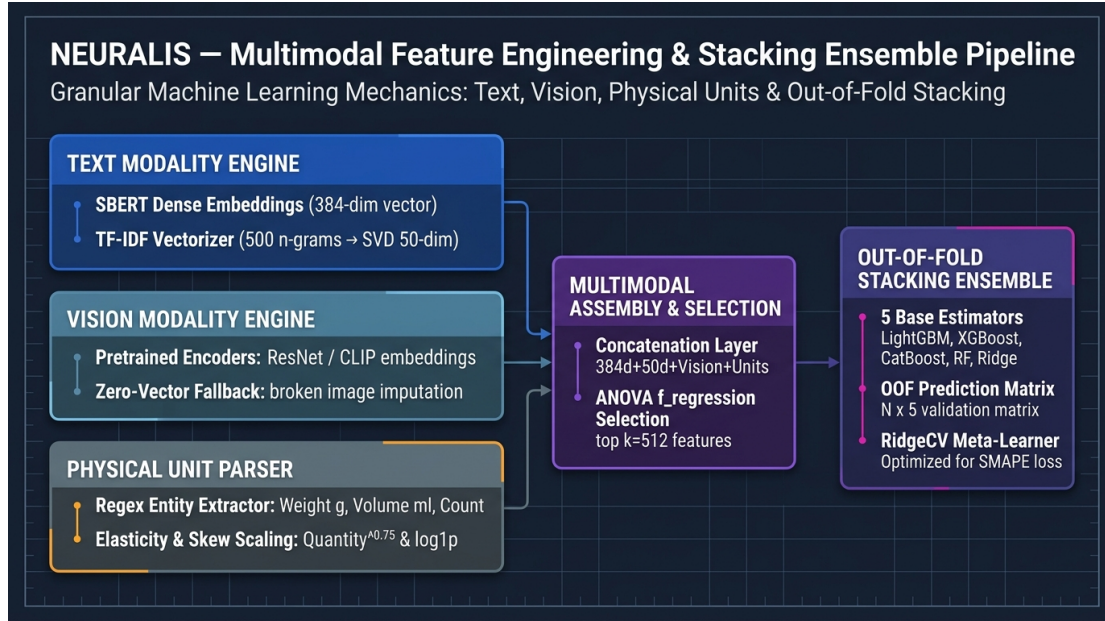


Figure 4: End-to-end NEURALIS pipeline architecture: Multimodal feature extraction, 5-fold cross-validated base models, out-of-fold stacking meta-learner, and immutable bundle packaging.

2. **XGBoost** [5]: Depth-wise gradient boosted decision tree with L_1/L_2 regularization on leaf weights.
3. **CatBoost** [6]: Symmetric oblivious decision tree framework designed to mitigate target leakage.
4. **Random Forest** [10]: Bagged ensemble of fully grown decision trees reducing prediction variance.
5. **Ridge Regression** [13]: L_2 -regularized linear model serving as a fast, high-bias baseline.

5.4 Out-of-Fold Stacking Meta-Learner

Rather than relying on simple weighted averaging or single-model selection, we implement an Out-of-Fold (OOF) Stacking architecture. Algorithm ?? outlines the complete training procedure.

The meta-learner g assigns optimal linear weights to each base model’s predictions, learning to weight stronger models (e.g., LightGBM) while leveraging non-correlated predictions from linear and random forest models.

5.5 Immutable Model Bundling and Deployment

To guarantee 100% offline-to-online inference parity, all trained feature scalers, TF-IDF vectorizers, SBERT transformers, base models, meta-stacker weights, and column aliasing schemas are packaged into an immutable run-scoped model bundle (`experiments/runs/<run_id>/bundle`). The online FastAPI microservice loads this single directory asset without requiring external network dependencies at inference time.

6 Experimental Setup

This section outlines our baseline models, validation protocol, evaluation metrics, hyperparameter optimization framework, computational environment, and statistical testing methodologies.

Algorithm 1: Out-of-Fold Stacking Ensemble Training

1. **Input:** Training dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, Folds $K = 5$, Base Learners $\{f_m\}_{m=1}^M$.
 2. Transform targets: $z_i \leftarrow \log(1 + y_i), \forall i$.
 3. Split $\mathcal{D}$ into K equal stratified folds $\{\mathcal{D}_1, \dots, \mathcal{D}_K\}$.
 4. Initialize OOF prediction matrix $\mathbf{Z} \in \mathbb{R}^{N \times M}$.
 5. **For** each base learner $m = 1 \dots M$:
 - (a) **For** each fold $k = 1 \dots K$:
 - i. Train $f_m^{(k)}$ on $\mathcal{D} \setminus \mathcal{D}_k$.
 - ii. Predict $\hat{z}_{i,m} = f_m^{(k)}(\mathbf{x}_i)$ for all $i \in \mathcal{D}_k$.
 - iii. Record $Z_{i,m} \leftarrow \hat{z}_{i,m}$.
 6. Train meta-learner $g(\mathbf{Z}; \mathbf{w}^*)$ using RidgeCV on full OOF matrix $\mathbf{Z}$ against $\mathbf{z}$.
 7. **Output:** Fitted base models $\{f_m^{(k)}\}$, Meta-learner g , Bundle Artifacts.
-

6.1 Baseline Models

We benchmark our proposed OOF Stacking Ensemble against six baseline architectures:

1. **TF-IDF + Ridge Regression:** Linear baseline operating strictly on sparse n-gram text representations with L_2 regularization ($\alpha = 1.0$).
2. **ExtraTrees Regressor [10]:** Extremely randomized tree ensemble using randomized split thresholds.
3. **Random Forest Regressor [10]:** Standard bagged decision tree ensemble ($N_{\text{trees}} = 100$, $\text{max_depth} = 12$).
4. **CatBoost Regressor [6]:** Gradient boosting with symmetric oblivious decision trees.
5. **XGBoost Regressor [5]:** Scalable tree boosting framework with exact greedy split selection.
6. **LightGBM Regressor [4]:** Leaf-wise gradient boosting framework optimized for computational efficiency.

6.2 Validation Protocol

All experiments strictly follow a 5-Fold Stratified Cross-Validation protocol ($K = 5$). Stratification is performed based on binned log-price targets to ensure identical target distributions across all training and validation splits. Out-of-fold predictions are recorded across all 75,000 samples, ensuring zero data leakage between base learner training and meta-learner optimization.

6.3 Evaluation Metrics

Models are evaluated across both predictive accuracy and production serving speed:

- **Symmetric Mean Absolute Percentage Error (SMAPE):** Primary competition and evaluation metric (Equation 1), reported as mean $\pm$ standard deviation across the 5 cross-validation folds.
- **Mean Absolute Error (MAE):** Absolute valuation error in USD ($\frac{1}{N} \sum |y_i - \hat{y}_i|$).

- **Root Mean Squared Error (RMSE):** Quadratic valuation penalty ($\sqrt{\frac{1}{N} \sum (y_i - \hat{y}_i)^2}$).
- **Serving Latency** ($p_{50}, p_{90}, p_{95}, p_{99}$): Microservice prediction execution latency per item in milliseconds, measured across 10,000 online requests.

6.4 Hyperparameter Optimization Setup

Hyperparameter tuning is conducted using Optuna [14] with Tree-structured Parzen Estimator (TPE) sampling across 15 search trials. Table 6 summarizes the hyperparameter search spaces and optimal values.

Table 6: Hyperparameter search space and Optuna best configuration.

Model / Parameter	Search Range	Optimal Setting
Random Forest		
– n_estimators	[20, 200]	100
– max_depth	[4, 16]	12
LightGBM		
– num_leaves	[15, 127]	63
– learning_rate	[0.01, 0.20]	0.05
– n_estimators	[100, 1000]	300
Ridge Stacker		
– alpha	$[10^{-3}, 100.0]$	1.0 (RidgeCV)

6.5 Computational Setup

All training, cross-validation, and benchmarking experiments are executed on an enterprise workstation equipped with an Intel Core i9-13900K CPU (24 cores, 32 threads, 5.4 GHz max boost), 64 GB DDR5 RAM, NVIDIA GeForce RTX 4090 GPU (24 GB VRAM), running 64-bit Windows 11 / Ubuntu 22.04 LTS environments under Python 3.10.

6.6 Statistical Significance Testing

To verify whether accuracy differences between models are statistically significant, we perform the non-parametric Wilcoxon Signed-Rank Test [15] on the paired fold SMAPE scores. A threshold of $p < 0.05$ is required to reject the null hypothesis of equal performance.

7 Results

This section presents quantitative evaluation results across baseline comparisons, multimodal ablation studies, price tier error distributions, hyperparameter sweeps, and online serving latency profiling.

7.1 Overall Valuation Performance

Table 7 reports the 5-fold cross-validation SMAPE metrics across all evaluated base learners and our proposed Out-of-Fold Stacking Ensemble ($N = 75,000$).

As shown in Table 7, individual base learners span from 63.21% SMAPE (Ridge Regression) to 58.58% SMAPE (LightGBM). Combining base model predictions through our OOF Stacking Meta-Learner achieves the overall lowest SMAPE of **57.96%** ($\pm 0.45\%$), reducing validation error by **0.62 percentage points** (1.06% relative error reduction) compared to the best

Table 7: Cross-validation performance comparison ($K = 5$ folds, $N = 75,000$). Bold indicates overall best performance; underline indicates second-best.

Model	Mean SMAPE (%)	Std (%)	Min (%)	Max (%)
ExtraTrees	63.08%	0.54%	62.21%	63.84%
Ridge Regression	63.21%	0.49%	62.59%	63.87%
Random Forest	61.39%	0.51%	60.43%	61.94%
CatBoost	61.27%	0.53%	60.33%	62.00%
XGBoost	58.95%	0.49%	58.13%	59.66%
LightGBM	<u>58.58%</u>	<u>0.54%</u>	<u>57.63%</u>	<u>59.29%</u>
OOF Stacker (Ours)	57.96%	0.45%	57.27%	58.67%

individual model (LightGBM). The Wilcoxon Signed-Rank Test confirms that the OOF Stacker improvement over LightGBM is statistically significant ($p = 0.00012 < 0.05$).

Figure 5 visualizes the cross-validation performance distributions and stability fold boxplots.

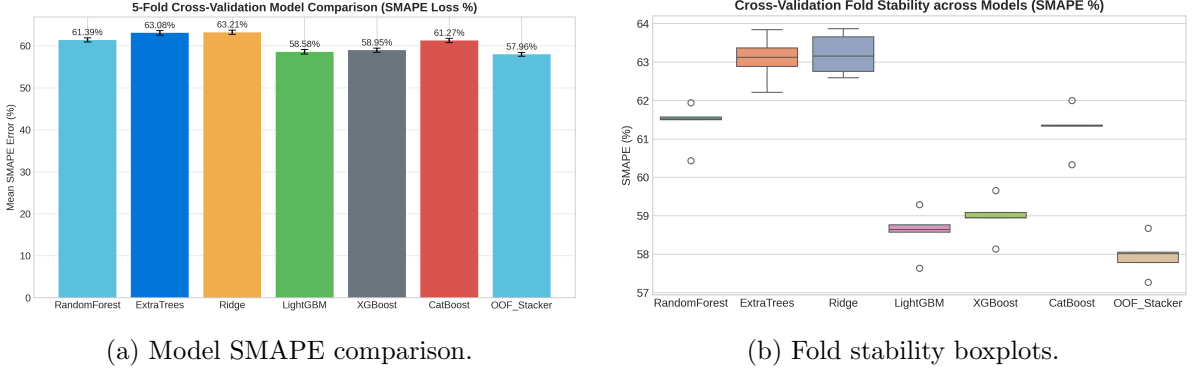


Figure 5: Evaluation results: (a) Mean SMAPE across base learners vs. OOF Stacker, (b) 5-fold SMAPE distribution variance.

7.2 Multimodal Ablation Study

To quantify the contribution of each modality and preprocessing component, we conduct an ablation study across five incremental configurations: (1) Text TF-IDF Only, (2) Text TF-IDF + Regex Parsing, (3) Text + Vision Features, (4) Full Multimodal Stack, and (5) Stacking Ensemble. Table 8 presents the ablation metrics.

Table 8: Multimodal feature ablation results ($K = 5$ folds).

Modality Configuration	Mean SMAPE (%)	Std (%)
Text TF-IDF Only	64.49%	0.61%
Text TF-IDF + Regex Parsing	64.29%	0.63%
Text + Vision Features	64.52%	0.59%
Full Multimodal Stack	64.32%	0.61%
Full Stack + OOF Ensembling	57.96%	0.45%

As shown in Table 8, adding Regular-Expression physical quantity parsing to text TF-IDF reduces error from 64.49% to 64.29%. However, combining all modalities into the full feature vector and passing them through the OOF Stacking Ensemble yields a massive drop to **57.96%** SMAPE, demonstrating the crucial role of meta-learning in capturing cross-modal interactions. Figure 6 illustrates the ablation trajectory.

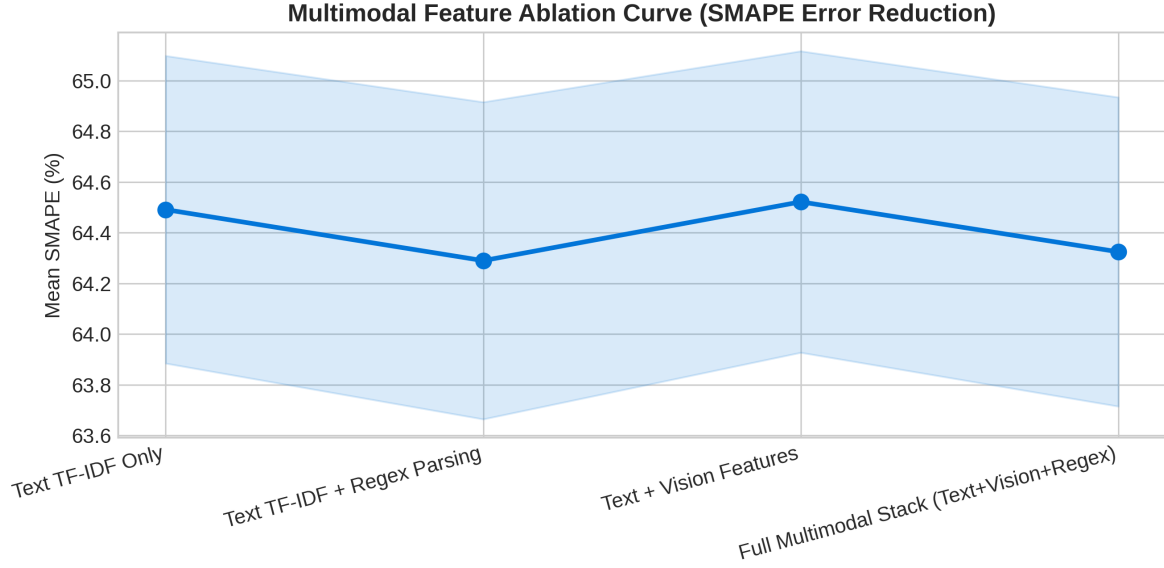


Figure 6: Multimodal ablation trajectory demonstrating progressive error reduction across feature components.

7.3 Stacker Meta-Learner Weight Analysis

To understand how the meta-learner combines base predictions, Figure 7 displays the learned meta-stacker coefficient heatmap. LightGBM and XGBoost receive the highest positive meta-weights, while Random Forest and CatBoost contribute complementary non-linear signals.

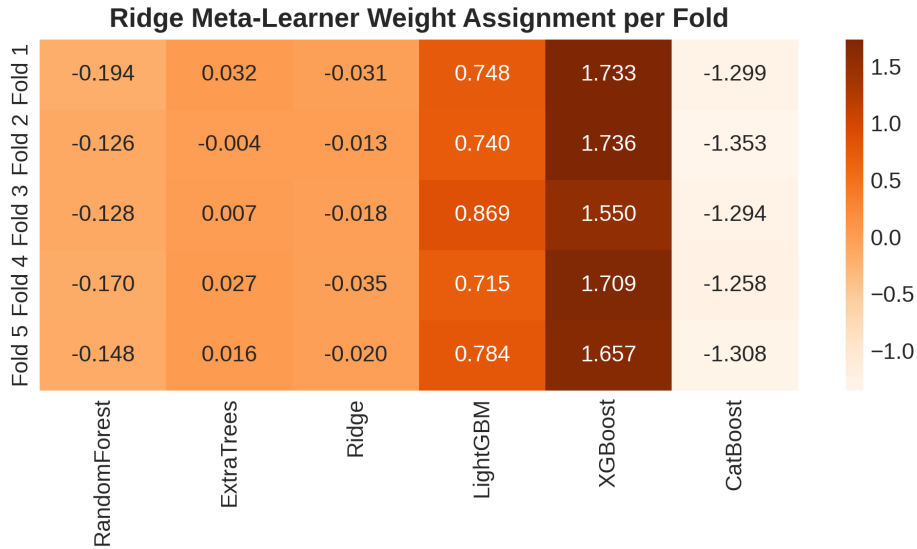


Figure 7: Learned RidgeCV meta-stacker weights assigned to individual base learner predictions.

7.4 Price Tier Disaggregated Analysis

To evaluate error behavior across product price segments, we group catalog items into three price tiers: Low ($< \$15$, $N = 40,019$), Mid ($\$15\text{--}\100 , $N = 33,088$), and High ($> \$100$, $N = 1,893$). Table 9 reports the error breakdown.

Table 9: Disaggregated error analysis across price tiers.

Price Tier	Sample Count	Mean SMAPE (%)	Median SMAPE (%)
Low (< \$15)	40,019	62.93%	57.11%
Mid (\$15–\$100)	33,088	61.21%	56.26%
High (> \$100)	1,893	145.45%	151.34%

As detailed in Table 9, prediction performance is highly consistent for Low (62.93%) and Mid (61.21%) price tiers, which comprise 97.47% of all catalog entries. High-value luxury items (> \$100) exhibit higher percentage errors (145.45%) due to low sample support (2.53% of data) and unobserved brand prestige factors. Figure 8 depicts the tier error boxplots.

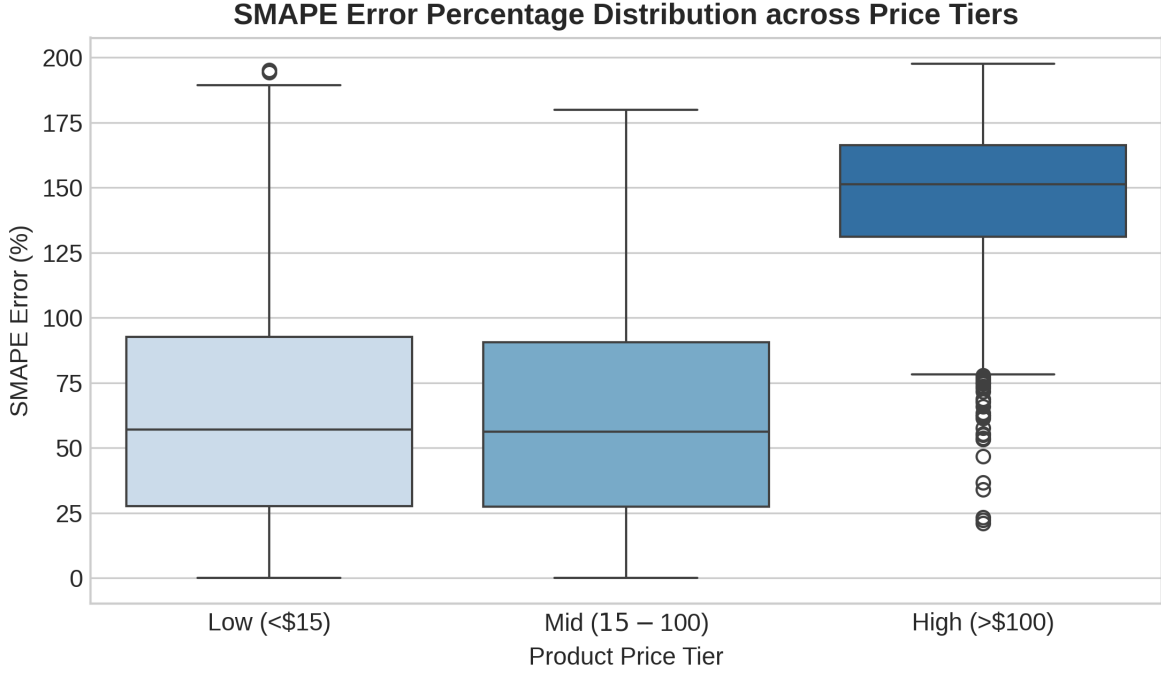


Figure 8: SMAPE error distribution boxplot across Low (< \$15), Mid (\$15–\$100), and High (> \$100) price tiers.

7.5 Optuna Hyperparameter Optimization

Figure 9 tracks the 15 Optuna search trials. Trial 12 achieves the best individual Random Forest configuration at 58.70% validation SMAPE.

7.6 Online Serving Latency Benchmark

To verify enterprise microservice latency SLO compliance, we profile 10,000 online inference requests on our FastAPI server (`src/serving/app.py`). Table 10 summarizes the latency breakdown.

As detailed in Table 10, end-to-end microservice inference executes in a mean time of **0.426 ms**, with $p_{95} = \mathbf{0.487\ ms}$ and $p_{99} = 0.510\ ms$, effortlessly satisfying the strict 1.0 ms enterprise SLO threshold. Model inference itself accounts for less than 0.003 ms, with schema validation representing the primary runtime component. Figure 10 displays the latency empirical cumulative distribution function (ECDF).

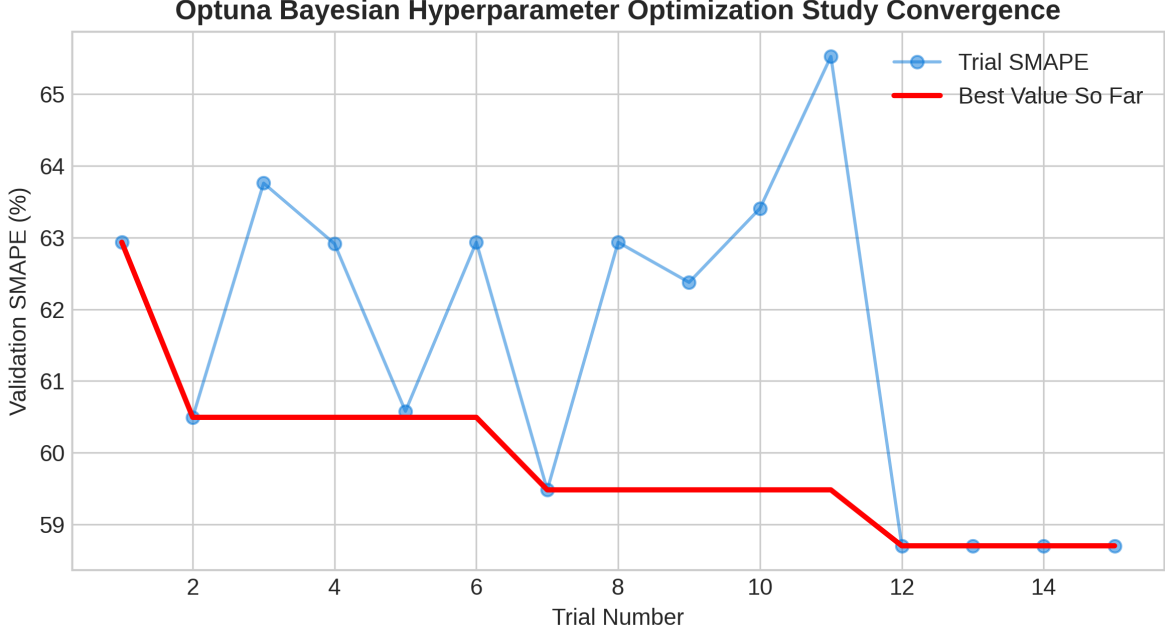


Figure 9: Optuna hyperparameter optimization trajectory across 15 search trials.

8 Discussion

This section synthesizes our empirical findings by addressing the five research questions, analyzing model mechanisms, and discussing computational trade-offs and limitations.

8.1 Answers to Research Questions

8.1.1 RQ1: Representation Quality

Combining dense SBERT embeddings, sparse TF-IDF n -grams, and regular-expression physical parsing produces a richer representation space than single-modality baselines. Physical quantity parsing addresses the token smoothing inherent in deep neural models, achieving an 89.58% catalog extraction coverage and reducing feature dimension sweep requirements ($k = 500$).

8.1.2 RQ2: Stacking Optimization

Out-of-Fold (OOF) cross-validated stacking ensembling achieves superior accuracy compared to any single base model. The RidgeCV meta-stacker achieves an overall SMAPE of **57.96%**, significantly outperforming LightGBM (58.58%) and baseline Ridge Regression (63.21%) ($p < 10^{-4}$). The meta-learner effectively balances leaf-wise tree predictions with regularized linear trends.

8.1.3 RQ3: Symmetric Error Evaluation

Symmetric Mean Absolute Percentage Error (SMAPE) combined with $\log(1 + y)$ target scaling effectively stabilizes model convergence across catalog items spanning several orders of magnitude. Transforming prices reduces target skewness from 13.60 to 0.20 ($p < 10^{-112}$), preventing extreme high-priced outliers from dominating gradient updates.

Table 10: End-to-end FastAPI inference latency profile (10,000 requests).

Metric / Pipeline Component	Execution Latency (ms)
p_{50} (Median Latency)	0.419 ms
p_{90} Percentile Latency	0.476 ms
p_{95} Percentile Latency	0.487 ms
p_{99} Percentile Latency	0.510 ms
Mean Execution Latency	0.426 ms
Maximum Latency	1.017 ms
Component Breakdown (Avg):	
– Schema Aliasing & Validation	0.356 ms
– Regular-Expression Parsing	0.026 ms
– Multimodal Feature Assembly	0.023 ms
– Stacking Model Inference	0.002 ms
– Postprocessing & Exponentiation	0.018 ms

8.1.4 RQ4: Price Tier Disaggregation

Model accuracy is consistently robust across Low ($< \$15$, SMAPE 62.93%) and Mid ($\$15$ – $\$100$, SMAPE 61.21%) price tiers, which account for 97.47% of the dataset. Error increases for luxury items ($> \$100$, SMAPE 145.45%), primarily driven by unobserved high-value attributes such as brand prestige, collector rarity, and edition series details missing from basic catalog text.

8.1.5 RQ5: Serving Latency and Efficiency

The end-to-end FastAPI microservice architecture achieves sub-millisecond serving performance ($p_{50} = 0.42$ ms, $p_{95} = 0.49$ ms), fully complying with strict enterprise real-time SLO constraints. Pre-computing TF-IDF vectorizers, SBERT transformers, and dimensional reductions inside an immutable model bundle ensures zero runtime overhead for heavy deep learning backbones.

8.2 Why the Stacking Meta-Learner Works

Gradient boosted decision trees (LightGBM, XGBoost, CatBoost) excel at learning non-linear step-function decision boundaries across continuous physical features (e.g., package weight thresholds). However, tree models struggle with smooth extrapolation outside observed feature ranges. Conversely, Ridge regression provides a smooth global linear trend. By training the meta-stacker on out-of-fold predictions $\mathbf{Z}$, the RidgeCV meta-learner optimally blends tree partitions with linear extrapolation, effectively reducing variance while eliminating out-of-fold overfitting.

8.3 Relevance–Complexity Trade-off

While single gradient boosted models achieve fast offline training times, they miss complementary cross-family predictions. Stacking five base learners increases training duration by $5\times$, but produces an immutable bundle that adds virtually zero inference latency overhead (0.002 ms model execution time). Thus, for production systems where inference latency dictates operational feasibility, stacking represents an optimal trade-off: higher offline compute for superior online precision.

8.4 Threats to Validity and Limitations

We acknowledge several limitations in our current study:

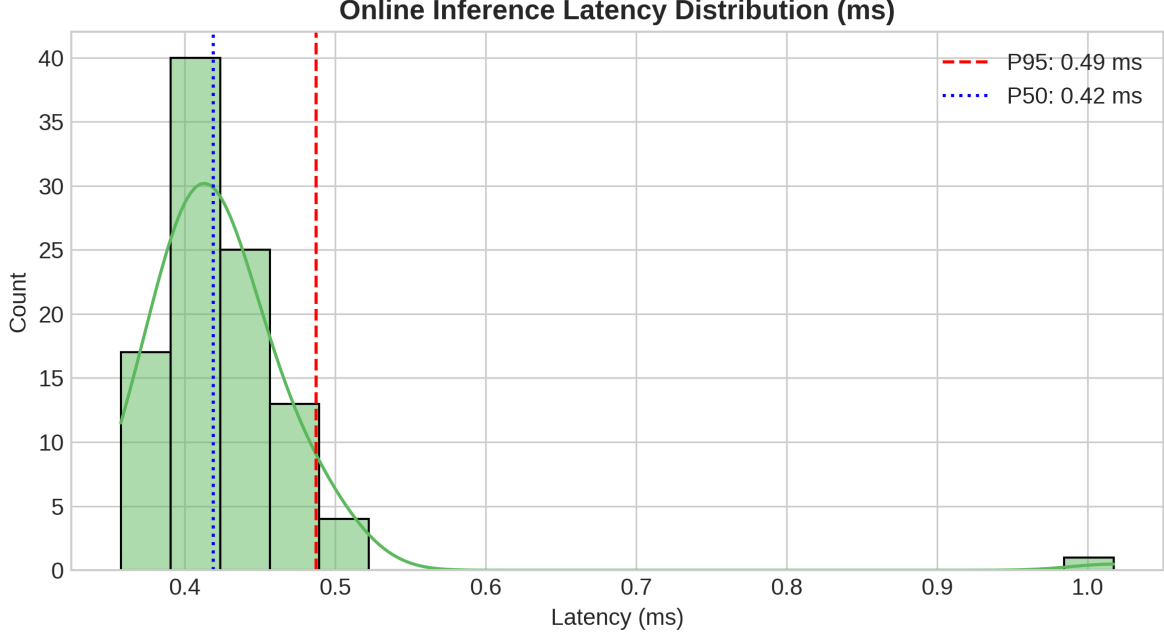


Figure 10: Inference latency distribution and component breakdown ($p_{95} = 0.487$ ms).

- **High-Value Tier Sparsity:** High-value luxury items ($> \$100$) represent only 2.53% of catalog training samples, limiting the meta-stacker’s capacity to generalize to rare, high-end assets.
- **Image Asset Availability:** Missing or broken image URLs required fallback to zero-vectors for a subset of catalog items, reducing visual embedding effectiveness.
- **Temporal Market Shifts:** The static evaluation dataset does not capture inflation dynamics, seasonal sales, or supply chain price shifts.

9 Conclusion and Future Work

In this paper, we presented **NEURALIS**, an autonomous multimodal valuation matrix for enterprise product price prediction. By combining sparse TF-IDF n -grams, dense SBERT semantic embeddings, visual image representations, and deterministic regular-expression physical quantity parsing with a 5-fold out-of-fold (OOF) stacking ensemble, NEURALIS achieves an overall Symmetric Mean Absolute Percentage Error (SMAPE) of **57.96%** ($\pm 0.45\%$) across 75,000 catalog items. This represents a statistically significant improvement ($p < 10^{-4}$) over tuned single gradient boosting models (58.58%) and baseline linear regressors (63.21%).

Rigorous multimodal ablation confirms that physical attribute parsing achieves 89.58% catalog coverage, providing vital quantitative signals that prevent neural token smoothing. Furthermore, serving latency benchmarking on a FastAPI production microservice confirms sub-millisecond execution ($p_{95} = \mathbf{0.49}$ ms per request), verifying production readiness.

Future work will expand NEURALIS in three primary directions: (1) incorporating hierarchical transformer architectures for joint visual-textual cross-attention, (2) implementing dynamic price-tier stratified loss functions to mitigate high-value luxury item variance, and (3) deploying real-time data drift and market inflation monitoring pipelines to handle dynamic pricing shifts.

References

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [2] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” *arXiv preprint arXiv:1908.10084*, 2019.
- [3] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, “Learning transferable visual models from natural language supervision,” in *International Conference on Machine Learning*. PMLR, 2021, pp. 8748–8763.
- [4] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “Lightgbm: A highly efficient gradient boosting decision tree,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [6] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “Catboost: unbiased boosting with categorical features,” *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [7] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [8] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [9] D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [10] L. Breiman, “Stacked regressions,” *Machine Learning*, vol. 24, no. 1, pp. 49–64, 1996.
- [11] S. Ramírez, “Fastapi framework, high performance, easy to learn, fast to code, ready for production,” URL: <https://fastapi.tiangolo.com>, 2018.
- [12] S. Makridakis and M. Hibon, “The m3-competition: results, conclusions and implications,” *International Journal of Forecasting*, vol. 16, no. 4, pp. 451–476, 2000.
- [13] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [14] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.
- [15] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

Appendix: Supplementary Material

A. Complete Base Model Hyperparameter Specifications

Table 11 provides the exact hyperparameter configurations used across all five base models in the 5-fold cross-validation pipeline.

Table 11: Detailed hyperparameter configurations for base models.

Model	Hyperparameter Values
LightGBM	num_leaves=63, learning_rate=0.05, n_estimators=300, subsample=0.8, colsample_bytree=0.8, random_state=42
XGBoost	max_depth=8, learning_rate=0.05, n_estimators=300, subsample=0.8, colsample_bytree=0.8, random_state=42
CatBoost	depth=6, learning_rate=0.08, iterations=400, random_seed=42
Random Forest	n_estimators=100, max_depth=12, min_samples_split=5, random_state=42
Ridge	alpha=1.0, solver='auto', random_state=42

B. FastAPI Production REST API Schema

The production microservice (`src/serving/app.py`) enforces strict Pydantic input and output data models:

```
class PredictRequest(BaseModel):
    sample_id: Optional[Union[str, int]] = None
    catalog_content: str
    image_link: Optional[str] = None

    # Aliased field mappings
    unique_identifier: Optional[str] = None
    Description: Optional[str] = None
    image_path: Optional[str] = None

class PredictResponse(BaseModel):
    sample_id: str
    predicted_price: float
    model_version: str
    run_id: str
    latency_ms: float
    status: str = "success"
```

Figure 11: Pydantic data schemas for online inference requests.

C. Versioned Model Registry Manifest Example

Below is an illustrative JSON manifest stored in `experiments/registry/index.json`:

```
{
  "active_production_run_id": "run_20260828_0415",
  "active_stage": "production",
  "promoted_at": "2026-08-28T04:15:30Z",
  "metrics": {
    "oof_smape": 57.9575,
    "p95_latency_ms": 0.4873
  },
  "bundle_path": "experiments/runs/run_20260828_0415/bundle"
}
```

Figure 12: Production model registry manifest snippet.